CN & SCALER · SESSION 6 · WEEK 6 OF 8

(Ab)using CDNs

Someone else's computers, closer to your users - and everything they can do while the traffic is passing through.

```
$ curl -s https://www.cloudflare.com/cdn-cgi/trace | grep colo
colo=BOM        # you just found out which building answered you
colo=MAA
```

Akamai · CloudFront · Fastly · Google · Broadpeak · Cloudflare · SSI · ESI · Railgun · BBR · anycast

Course Schedule

WEEK 1
Network programming 101

WEEK 2
Computer Networking Eagle Eye View

WEEK 3
Server Side Considerations for scale

WEEK 4
nginx deep dive

WEEK 5
Evolution of HTTP

WEEK 6
(Ab)using CDNs

WEEK 7
Building for failures

WEEK 8
Streaming video at large scale

EARLY HTTP/1.1 · ASSIGNMENT · DUE BEFORE SESSION 7

Build a calculator that stays on the line

The task

any language, no framework, just a socket

```
GET /add?a=2&b=3    ->  200   5
GET /sub?a=10&b=4   ->  200   6
GET /mul?a=6&b=7    ->  200   42
GET /div?a=9&b=3    ->  200   3

GET /div?a=1&b=0    ->  400
GET /add?a=x&b=3    ->  400
GET /pow?a=2&b=8    ->  404
POST /add           ->  405
GET /add  (no Host)   ->  400
```

That is the entire feature set. You could write it in an afternoon, and the arithmetic is not the point.

Don't read 02-http11/server11.py until you have failed at least once.

How I will mark it

one socket. every request.

```
s = socket.create_connection(("localhost", 8080))

  GET /add?a=2&b=3      -> 200   5
  GET /sub?a=10&b=4     -> 200   6
  GET /mul?a=6&b=7      -> 200   42
  GET /div?a=1&b=0      -> 400
  GET /pow?a=2&b=8      -> 404
  POST /add             -> 405

  socket still open: True
  1 TCP handshake, 6 responses
```

If the socket dies before I am done, you have written a 1996 server.

The part that is actually hard. Keeping the connection open forces a question HTTP/1.0 never asked you: **where does this request end and the next one begin?** While you were hanging up, the answer was "at EOF", for free. Now you must consume exactly Content-Length bytes and not one more — byte n+1 belongs to somebody else.

Stretch, all optional: honour Connection: close · an idle timeout you can defend · chunked encoding · take all six at once and answer in order, which is pipelining.

▶ Now you write the spec

Track 1 — the server

```
$ ./bserve ./www 9000
```

accept a TCP connection
read one binary request frame
map the path to a file under a root
reply: status, headers, the bytes
404 if it is not there
400 if the frame is malformed
and keep the connection open

Track 2 — the client

```
$ ./bcurl -v localhost:9000/index.html
```

build the binary request frame
read the response, body to stdout
-v hexdumps every frame
exit non-zero on 4xx / 5xx

and never open a second connection

The bit in the middle is the actual project

A fixed-size frame header — you pick the fields and the widths, and you defend them. HTTP/2 chose 24 / 8 / 8 / 31. Why?

Headers: number the ten names you actually send, length-prefix the rest — HPACK's first two mechanisms, in an evening.

And one line you may not skip: a receiver meeting a frame type it does not know MUST skip it cleanly. That is how you leave room for a version 2.

What you hand in

1. The spec. Two pages. Enough for a stranger.
2. Your program.
3. An annotated hexdump of one complete request and response.

If you cannot annotate your own bytes, the spec is not finished.

In pairs: one server, one client, and the only thing that crosses between you is the spec. A client that only works against your own server is an implementation, not a protocol.

Upcoming Quiz Schedule

Quiz 1

21th Sep, Monday

(Binayak will announce timings)

Assignment submission

25th Sep, Friday

(during class time)

Quiz 2

25th Sep, Friday

WHERE WE ARE

Eight weeks, and today the machine stops being yours

Week 1	Network programming 101
Week 2	Eagle-eye view: OSI, SS7, protocols
Week 3	Server-side considerations for scale
Week 4	nginx deep dive
Week 5	Evolution of HTTP: 1.0 → QUIC
Week 6	**(Ab)using CDNs**
Week 7	Building for failures
Week 8	Video streaming at large scale

Weeks 1–5 made one origin fast. Today we put a network in front of it. Week 7 asks what happens when a piece of that fails, and Week 8 pushes the hardest traffic — video — through all of it.

AGENDA

Six questions about a box in the middle

01 **Why the edge exists**

The speed of light is a fixed budget, and your origin spends all of it.

02 **What a CDN actually is**

A reverse proxy — Session 4's nginx — rented in 300+ cities. Who the players are.

03 **Caching, including the uncacheable**

cf-cache-status end to end. SSI → ESI → Ajax → Railgun. Poisoning and deception.

04 **Moving bytes faster**

BBR, connection pooling, multiplexing to the origin.

05 **Everything that isn't speed**

WAF, rate limiting, DDoS, API gateway, and the trust you hand over.

06 **Cloudflare, hands-on**

Demos you can run now with no account; dashboard and Workers..

▶ The speed of light is the one number you cannot optimise

```
$ python3 bench/01_physics.py

light in vacuum   299,792 km/s
light in fibre    204,218 km/s   # glass slows it to ~2/3 (index 1.468)
                  = 4.90 ms per 1000 km, one way
```

10 ms

Mumbai → Chennai and back, in perfect straight glass. Measured ping: 23 ms.

126 ms

Mumbai → Ashburn (us-east-1) and back, floor. You cannot go under this. Ever.

47%

faster if the glass were hollow. Microsoft is deploying hollow-core fibre now — but it is not on your path today.

A round trip to Virginia costs 126 ms before a single byte of yours is processed. TLS 1.3 needs three of those round trips to first byte. That is ~380 ms of nothing, and no faster CPU touches it.

great-circle distance · fibre group index 1.468 · Microsoft Azure hollow-core fibre, Sep 2025

One handshake, not two

TCP + TLS 1.3 + HTTP/2

three round trips, cold

```
SYN   ->
      <-  SYN/ACK
ACK   ->                        1 RTT
      ...nothing has been said yet.

ClientHello ->    (ALPN rides here)
            <-    ServerHello …
Finished    ->                  2 RTT

GET   ->
      <-  200                   3 RTT
```

With TLS 1.2 it was four.

QUIC + HTTP/3

two round trips, cold

```
Initial  ->   TLS ClientHello inside
              a QUIC CRYPTO frame
         <-   Handshake           1 RTT

GET      ->
         <-   200                 2 RTT
```

RFC 9001 §2.1: there is no transport handshake and then a crypto handshake. There is one handshake.

And TLS is not optional. There is no unencrypted QUIC.

```
$ python3 05-http3/handshake_race.py --rtt 150

                                          measured   round trips
  TCP + TLS 1.3 + h2 (new connection)      456.6 ms          3.0
  QUIC + HTTP/3 (new connection, 1-RTT)    319.8 ms          2.1
```

Layering is how we make systems comprehensible — and every layer boundary is a place you may pay a round trip.

◂ S2: BBR vs the TCP sawtooth

▸ Distance charges you twice: once in latency, once in loss

From an Akamai edge, 2010	RTT	loss	throughput	4 GB download
local, < 100 mi	1.6 ms	0.6%	**44 Mbps**	12 min
regional, 500–1,000 mi	16 ms	0.7%	4 Mbps	2.2 hrs
cross-continent, ~3,000 mi	48 ms	1.0%	1 Mbps	8.2 hrs
multi-continent, ~6,000 mi	96 ms	1.4%	**0.4 Mbps**	20 hrs

Throughput ≈ MSS / (RTT × √loss)

Mathis et al., 1997. A single TCP flow's ceiling falls with RTT and with the square root of loss — and long paths have more of both. The far edge is 110× slower, not 60× closer.

Why one flow isn't enough

Session 5's fix — HTTP/2 multiplexing, many streams on one connection — and BBR, next section, both exist to claw back this exact loss.

Nygren, Sitaraman & Sun, "The Akamai Network", ACM SIGOPS OSR 44(3), 2010, Table 1

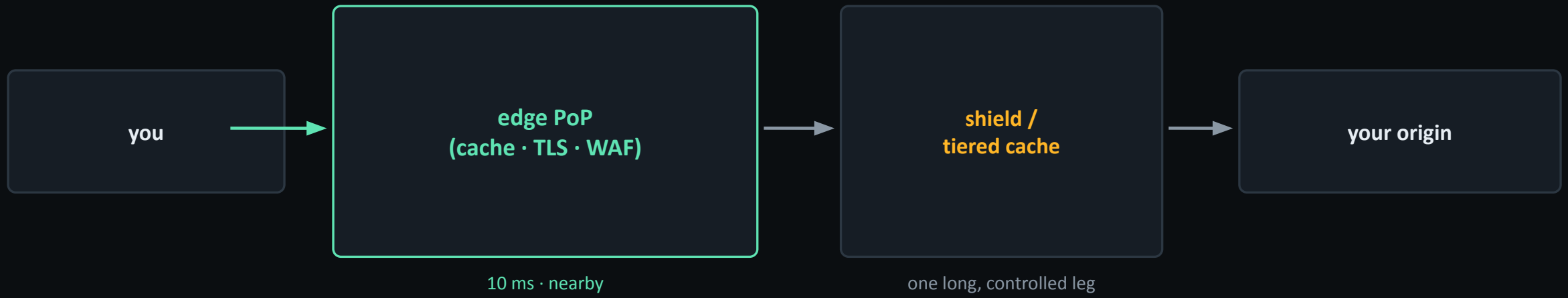
▶ The edge saves handshakes for free. Only the cache saves the round trip.

`you ——10 ms—— edge (Mumbai) ——194 ms—— origin (Washington)  ·  TLS 1.3 on every hop  ·  bench/02_split_tcp.py`

what the user waits for	content	TTFB	vs direct
direct to origin, new connection	dynamic	**621 ms**	3.0 RTT
direct to origin, connection kept alive	dynamic	208 ms	1.0 RTT
edge with no origin pool (just a longer proxy)	dynamic	625 ms	3.1 RTT
edge, warm origin pool, new user connection	dynamic	233 ms	1.1 RTT
edge, warm pool, connection kept alive	dynamic	209 ms	≈ direct
edge, cache HIT (origin never touched)	static	**34 ms**	0.2 RTT
edge, cache HIT, connection kept alive	static	**11 ms**	0.1 RTT

A cache miss through the edge is no faster than going direct — sometimes slower. The edge earns its place two ways: it keeps warm connections to your origin (handshakes), and, when it can cache, it deletes the origin round trip entirely (194 ms → 0).

▶ A CDN is Session 4's reverse proxy, rented in 300+ cities



Nothing here is new

- the reverse proxy is nginx — Session 4
- caching is proxy_cache — Session 4
- the pool is upstream keepalive — Session 3
- anycast puts one IP in every city — Session 2's addressing

Squid did reverse-proxy caching in 1996. A CDN is this, multiplied by geography and sold as a service.

What geography adds

- the edge is near the user, so the 3-RTT handshake is cheap
- the origin leg is long but the CDN owns both ends of it — its own transport, its own pooling
- a shield / tiered-cache layer means many edges share one origin fetch, not N
- one anycast address, announced from every city at once

Which cache holds which URL? Get this wrong and every upgrade is a cold cache

```
$ python3 bench/06_consistent_hash.py     # 100,000 URLs, 10 caches → add an 11th

scheme                              URLs that move     busiest vs mean
hash(url) % N                              90.9%              1.02x
consistent hash, 1 point/server             3.3%              2.93x
consistent hash, 160 points/server          8.2%              1.14x
```

Modulo hashing is a trap

Add one server and hash(url) % N moves ~90% of URLs to a different cache. A capacity upgrade becomes a near-total cache flush — and a stampede on your origin at the worst moment.

Consistent hashing (Karger et al., 1997)

Only the new server's fair share moves — ~1/11. Virtual points even out the load. This paper is what Akamai was founded on;

nginx ships it as `hash $request_uri consistent;` (160 points/server, ketama-compatible).

The same idea recurs all week: choosing an owner without a central registry — DHTs, sharded caches, this. Add capacity without moving the world.

Karger, Lehman, Leighton, Levine, Lewin, Panigrahy, STOC 1997 · nginx ngx_http_upstream_hash_module.c (peer->weight × 160)

▶ Six CDNs — and the software you already know, running inside them

CDN	born	runs on / notable for	you met it in
Akamai	1998	own “ghost” edge servers; ESI (2001); FastTCP→BBR	consistent hashing
Amazon CloudFront	2008	AWS edge; BBR to viewers since 2019 (+22%)	S2 congestion
Fastly	2011	a heavily-modified Varnish + H2O; instant purge	S3/S4 Varnish
Google (Media CDN)	2022	YouTube's edge, opened up; QUIC/BBR everywhere	S5 QUIC · BBR
Broadpeak	born of video	operator/telco CDN for streaming; on GKE too	S8 next week
Cloudflare	2010	was nginx+LuaJIT (FL1) → Rust/Oxy (FL2, 2025)	S4 nginx → Pingora

The software layer is Sessions 3 & 4

Apache · HAProxy · Varnish · nginx — the reverse proxies from the scaling week — are what most CDNs were built on. Fastly is Varnish. Cloudflare was nginx, and outgrew it: the customer edge went nginx+LuaJIT (FL1) → Rust on Oxy (FL2) in Oct 2025 (−10 ms, +25%), and the origin-facing proxy went to Pingora in 2022 — the same “outgrow your proxy” story the last two sessions left as a thread.

vendor blogs & docs, 2013–2026; H2O = Kazuho Oku; Pingora & FL2 = Cloudflare eng. blog

DEMO · no account

▶ One address, announced from 330+ cities — which one did you get?

```
# every Cloudflare-proxied site answers this path
$ curl -s cloudflare.com/cdn-cgi/trace
ip=203.0.113.9
colo=BOM          # ← Mumbai (IATA code)
http=http/1.1  tls=TLSv1.3
kex=X25519MLKEM768   # post-quantum

# which 1.1.1.1 resolver answered?
$ dig +short CHAOS TXT \
        id.server @1.1.1.1
"BOM"
```

colo is an airport code

BOM Mumbai · DEL Delhi · MAA Chennai · SIN Singapore · LHR London · IAD Ashburn. It is the CDN telling you which of its buildings your packets reached — chosen by BGP anycast, not by you.

The same trick is in every response

cf-ray: 8c1a…-BOM — the suffix is the colo. Every Cloudflare response carries the PoP that served it, which is how you debug “why is this user slow / stale”.

Run it on the classroom wifi, then ask someone on mobile data to run it. Same domain, same IP, different colo. That is anycast — Session 2's addressing, put to work.

cloudflare.com/cdn-cgi/trace (live, Sep 2026, colo=IAD from this build host) · 1.1.1.1 troubleshooting docs

▶ By default a CDN caches by file extension — and HTML is not on the list

What Cloudflare caches with no config

68 static extensions: css js png jpg webp woff mp4 pdf zip …

NOT cached: HTML, JSON, or anything with

Set-Cookie, or Cache-Control: private / no-store / no-cache / max-age=0

Default edge TTL when the origin is silent:

200/301 → 2 hrs · 404 → 3 min · others: not cached

The cache key is the question

Default key = scheme + host + path + query.

Everything else — cookies, most headers — is NOT in the key. Two users with different cookies share one cached object.

That is correct for app.css and catastrophic for /account. Vary:, or a custom key, is how you say “these differ”. Session 5's Vary, at the edge.

```
$ curl -sI edge/static/app.css | grep -i x-cache
x-cache: MISS          # first visitor pays the origin round trip
x-cache: HIT           # everyone after is free — origin never touched

$ curl -sI edge/page/home | grep -i x-cache
x-cache: DYNAMIC       # HTML: not an eligible extension, straight to origin
```

▶ cf-cache-status is a small language. Learn to read all of it.

status	what happened	origin hit?	your callback
MISS	eligible, not in cache — fetched from origin	yes	—
HIT	served from cache, still fresh	no	the whole point
EXPIRED	was cached, TTL passed, re-fetched	yes	—
REVALIDATED	TTL passed, origin said 304 — cache reused	cheap 304	S5 ETag / If-None-Match
UPDATING	stale served now, refresh in background	async	S4 proxy_cache_use_stale
STALE	origin is down — serve stale anyway	failed	S7: failing gracefully
DYNAMIC	not eligible — never looked in cache	yes	the default for HTML
BYPASS	eligible, but origin said no-store	yes	S5 Cache-Control

One header, eight states. REVALIDATED is Session 5's conditional request saving bandwidth; STALE is next week's whole topic — the cache is the last thing standing when the origin dies.

▸ Pages got personal. Four tricks to cache them anyway.

SSI Server-Side Includes

`NCSA HTTPd 1.0a1, 1994`

Holes in an HTML file, filled by the server before sending. Predates CGI. nginx still ships it — Session 4.

ESI Edge-Side Includes

`Akamai + Oracle, W3C Note 2001`

The same holes, but filled at the EDGE. Cache the shared shell everywhere; assemble per-user at the PoP.

Ajax Web 2.0

`~2005`

Push the holes all the way to the browser. Cache the body on the CDN; fetch the personal bits async.

Railgun delta to origin

`Cloudflare 2013–2024`

Give up on caching the dynamic page; send only the BYTES that changed since last time.

One arc: move the personalisation outward — server → edge → browser — so the big shared part can be cached, and only the small personal part is ever recomputed. Each step shrinks what the origin has to do.

ESI shrinks the origin's job by 98% — and the user downloads the same page

```
# the origin sends ONE shell, cacheable by everyone, with two holes:
  <header><esi:include src="/fragment/user"/> <esi:include src="/fragment/cart"/></header>
  Cache-Control: s-maxage=300      Surrogate-Control: content="ESI/1.0"

$ python3 bench/04_esi_bytes.py     # 100 logged-in users, same page
100 users; pages identical per user: yes (39,676 B each)
```

	origin requests	origin → edge bytes	bytes to users	edge cache
A · origin assembles the page	100	3,984,200	3,992,400	DYNAMIC ×100
B · ESI, edge assembles	201	**83,454**	3,994,793	HIT ×99, MISS ×1

Read the trade carefully

Origin bytes fell 98% (the shell is fetched once, not 100 times). Bytes to the user are unchanged — ESI does not shrink the download. And origin requests went UP (100 → 201), because each personal fragment is its own tiny fetch. You move load off the origin's page-rendering, onto the edge's assembly. Whether that wins depends on which is your bottleneck.

bench/04_esi_bytes.py — the two pages are asserted byte-for-byte identical before counting · also cloudflare/worker-esi (run in workerd)

▶ Railgun's idea came back as an internet standard in 2025

```
$ python3 bench/05_delta.py    # a ~24 KB news page, regenerated every minute

compress the new page…              bytes     % of raw
  gzip -6 (no dictionary)           4,276        17.7%
  deflate, dict = page 1 min ago      598         2.5%
  zstd, dict = page 1 min ago  (dcz) 436          1.8%
  zstd, dict = page 1 HOUR ago      1,277         5.3%
```

Railgun (Cloudflare, 2013–2024)

A daemon inside your origin + a sender at the edge kept the last version of each page on both ends, and sent only the delta. “Caching down to the individual bytes.” Retired Jan 2024 — the network got fast enough, and a standard arrived.

RFC 9842 — Compression Dictionary Transport (Sep 2025)

The same idea, standardised, browser ↔ server: the version you already have becomes the dictionary for the next one.

Available-Dictionary: <sha-256> · Content-Encoding: dcb / dcz

This is Session 5's “why not gzip the headers” — dictionaries leak, so it is same-origin, HTTPS-only, hashed.

A one-minute-old copy turns 24 KB into 436 bytes — ~10× past gzip. The catch: the dictionary must be recent and identical on both ends, which is exactly what Railgun kept, and what RFC 9842 names by hash.

Cloudflare Railgun press release 2013 & deprecation 2023 · RFC 9842 (dcb/dcz), RFC 9841 (shared brotli), Sep 2025

▶ A shared cache is a shared weapon: poisoning and deception

```
$ curl -H 'X-Forwarded-Host: evil.example' edge/page/landing  # ONE attacker request
<script src="https://evil.example/app.js"></script>

$ curl edge/page/landing                                    # every visitor, next 60 s
<script src="https://evil.example/app.js"></script>   x-cache: HIT
```

Cache poisoning (Kettle, 2018)

The origin reflects a header (X-Forwarded-Host) into the page. That header is NOT in the cache key. One malicious request caches a hostile page; everyone else is served it. Fix: drop the unkeyed header at the edge, add it to the key, or Vary on it.

Cache deception (Gil, 2017)

Trick the victim into loading /account/settings/fake.css. The edge sees ".css", caches the response — which contained the victim's private page. Attacker then fetches the same URL from the cache. Fix: don't cache by extension alone; honour the origin's content-type and no-store.

Both bugs are the cache key disagreeing with reality — the same root as the personalisation problem, turned into an exploit. A cache is shared state, and shared state is an attack surface.

J. Kettle, "Practical Web Cache Poisoning", 2018 · O. Gil, "Web Cache Deception Attack", Black Hat USA 2017 · demos/poison.sh

▶ BBR — and it is Session 2's sawtooth

The correction

The source doc: “BBR algorithm by Google … relooked at TCP retries, congestion and the famous saw-tooth.”

Bottleneck Bandwidth and Round-trip propagation (Cardwell et al., Google, 2016). The saw-tooth is Session 2's slide.

How it differs

Loss-based (CUBIC): fill the pipe until a packet drops, then halve — the saw-tooth. On long, lossy CDN paths it mistakes distance for congestion.

BBR: measure the actual bottleneck bandwidth and the min RTT, pace to that. Loss ≠ congestion signal.

deployed	who / where	result
Google B4 + YouTube, 2016	the paper	2–25× CUBIC throughput; median RTT −53% globally
Amazon CloudFront, 2019	edge → viewer	up to +22% throughput; some US mobile +100%
Akamai, Nov 2019	media delivery	BBR on >80% of end-user connections
measured across the web, 2023	top-10k sites	BBR now the most-deployed CCA; CUBIC ~7%

Only the sender changes — no new protocol, no client, no router. That is why a CDN can just switch it on for you: it owns the sender on the long leg back to your origin, and the one out to the user.

Cardwell, Cheng, Gunn, Yeganeh & Jacobson, ACM Queue 14(5), 2016 · AWS 2019 · Akamai 2019 · SIGCOMM 2024 CCA census

▶ Reuse the origin connection, or pay the handshake every time

```
$ python3 bench/03_pool.py   # 300 visitors, edge↔origin 50 ms, TLS 1.3, origin idle-close 1 s

edge configuration      origin reqs   new conns    reuse     mean TTFB
no pool                        300          300     0.0%        160 ms
32 worker pools                300          164    45.3%        112 ms
8 worker pools                 300           52    82.7%         74 ms
1 shared pool                  300            4    98.7%         57 ms
```

Why the shared pool wins

A pooled connection opened by one worker can only be reused by that worker. Split the pool across many workers and each sees too few requests to keep its connections warm before the origin closes them. This is nginx's per-worker pool vs Pingora's shared pool — Session 4's Cloudflare thread.

Cloudflare's Pingora, in production

Moving from per-worker nginx pools to one shared Pingora pool took one big customer's reuse from 87.1% → 99.92% — a 160× drop in new origin connections, and "434 years of handshake time saved every day."

Same shape as the bench, three orders of magnitude bigger.

"Amortise the setup" — Session 3's first closing idea — now spanning a continent. The setup you're amortising is a 2-RTT TLS handshake to your origin.

bench/03_pool.py · Cloudflare, "How we built Pingora", Sep 2022 (87.1%→99.92%, 434 years/day)

◂ S5: h2 multiplexing

▶ Fewer connections, fewer origin fetches — the same idea at two layers

Multiplexing (Session 5, applied to the origin)

HTTP/2 puts many streams on one connection; the source doc's line is “we've discussed multiplexing already.” A CDN uses it on both legs — many user streams in, a few multiplexed streams back to origin.

But note the default: most CDN→origin hops are still HTTP/1.1. cloudflared's http2Origin defaults to false; you turn it on. Pooling (last slide) is what actually saves you there today.

Tiered cache / origin shield

Without it: 300 edges each MISS → 300 requests hit your origin for the same object. A built-in thundering herd — Session 3 & 4's cache_lock, globally.

With it: edges fetch from a regional parent; the parent fetches from origin once. Your origin sees one request, not 300. Cloudflare: Tiered Cache / Argo. nginx did the single-flight version with proxy_cache_lock.

```
# request collapsing at one edge (bench/harness, --collapse): 50 users, cold cache, 1 object
  without collapsing   50 origin fetches    # every miss stampedes
  with collapsing       1 origin fetch      # 49 wait for the first, then all HIT
```

Collapsing (one PoP) and tiered cache (across PoPs) are the same move at two scales: never fetch the same object twice at once. It is why a CDN protects your origin during a traffic spike instead of amplifying it.

▶ All your traffic already passes through it. That changes what it's for.

“All requests are coming to the CDN anyway…” — the source doc, three times in a row

WAF
SQLi, XSS, path traversal — inspected before your app sees them

Rate limiting
cap requests per client, per path, before they cost you anything

DDoS
absorb a 31.4 Tbps flood with capacity you could never buy

API gateway
authenticate, add/strip headers, route, fall back — at the edge

The biggest shift in what a CDN is: it started as a cache and became the security perimeter, because the perimeter is wherever all the traffic converges — and for you, that is the edge. Speed got you to adopt it; control is why you can't leave.

▶ A WAF is a denylist of spellings. The bug still lives in your origin.

```
$ bash demos/waf.sh     # origin builds SQL with string formatting; edge --waf on

  q="' OR 1=1 --"           → blocked by edge rule sqli-tautology
  q="'/**/OR/**/1=1--"      → 5 rows leak   # comments instead of spaces; regex misses
  (edge --waf full, strips comments first)
  q="'/**/OR/**/1=1--"      → blocked        # normalisation caught it…
  q="' --"                  → 5 rows leak   # …but no OR, no UNION, no digits to match
```

The arms race is normalisation

A WAF matches patterns in text, so the game is how much decoding it does first (raw → URL-decode → strip comments → …) vs how many ways the attacker can spell the same attack. You never win it outright.

So what is it FOR?

Buying time and blocking the mass-scanned, known payloads (Log4Shell, the OWASP top patterns) — a real, useful layer. The actual fix is in the origin: bind parameters. demos/waf.sh ends by turning on --safe, and every bypass returns “no results.”

demos/waf.sh · edge/waf.py (5 rules, 3 normalisation levels) · Cloudflare Free Managed Ruleset (all plans); OWASP Core needs Pro

▶ “100 per 10 seconds” has at least five different meanings

```
$ python3 bench/07_ratelimit.py    # limit 100 / 10 s, simulated time

algorithm                   burst at the window edge     normal page load
fixed window                200 in one second   !!         all pass
sliding log (exact)         100                            all pass
sliding window (estimate)   104                            all pass
token bucket                109                            all pass
leaky bucket, burst=0        10                !!         97% refused
```

The two failure modes

Fixed window: 100 at 9.999 s + 100 at 10.001 s = 200 in a blink. Cloudflare's free plan only offers this 10 s window.

Leaky bucket with no burst: mathematically pure, refuses a real browser that opens 30 requests at once. Why nginx's limit_req has a burst parameter — Session 4's self-study.

Cloudflare's real answer (2017)

The sliding-window estimate: weight the previous window by how much still overlaps. Two counters per client. Measured across 400M requests: only 0.003% wrongly allowed or blocked. Cheap and close enough.

Counts are per data-center, not global — a botnet spread across 330 cities gets 330 budgets. The algorithm and the geography both matter.

bench/07_ratelimit.py · edge/ratelimit.py (nginx limit_req excess formula) · Cloudflare, “Counting things”, Jun 2017

▶ The one thing you genuinely cannot do yourself: absorb a flood

31.4 Tbps

largest DDoS on record (Cloudflare, Dec 2025). Aisuru botnet, 1–4M devices. Lasted 35 seconds.

auto

mitigated automatically, no human in the loop. You cannot buy 31 Tbps of inbound to soak this.

~500 Tbps

Cloudflare's total network capacity, across 330+ cities — the pool the flood lands in.

Why only a CDN can do this

A volumetric flood is won with more capacity than the attacker, spread across enough locations that anycast splits the flood across all of them (Session 2). Your origin has one pipe; the CDN has hundreds. This is the purest case of “rent capacity you could never own” — and it is why sites move behind a CDN the day after an attack, not before.

The catch (Session 7 preview)

The flip side of one network absorbing everyone's attacks is one network being everyone's single point of failure. June 2021: a Fastly config bug took 85% of its network to errors — Reddit, Spotify, the UK government, gone at once. Concentration cuts both ways.

Cloudflare Q4-2025 DDoS report (31.4 Tbps, Aisuru-Kimwolf, 35 s, auto-mitigated) · network capacity ~500 Tbps / 330+ cities

◂ S5: JWT is base64url

▶ Authenticate, scrub, route — at the edge, before the origin is touched

```
$ bash demos/gateway.sh    # edge verifies the JWT; origin only trusts the edge

  no token                    → 401 edge: missing bearer token  # origin never sees it
  valid JWT                   → {"user":"alice","orders":[…]}  # edge sets X-User
  expired JWT                 → 401 edge: expired
  alg:none forgery            → 401 edge: alg 'none' refused    # never let the token pick
  client sends X-User: admin → ignored — edge overwrites the identity it derived
  attacker hits origin IP     → {"orders":[EVERYTHING]}   !! the edge is bypassed
```

The gateway is only real if the origin can't be reached directly

The last line is the whole point: put auth at the edge and leave the origin open, and an attacker just skips the edge. The fix is mutual TLS (authenticated origin pulls) or a Tunnel — the origin only answers clients holding the edge's certificate. demos/gateway.sh ends by turning that on: the direct request gets “connection reset”, the edge still works. Header hygiene too: the origin leaks X-Powered-By: PHP/5.4.45; the edge strips it.

demos/gateway.sh · edge/jwtlite.py (HS256, WebCrypto) · cloudflare/worker-gateway (same logic, run in workerd)

DEMO · no account

▶ Six things every one of you can run on a laptop, no account

```
# 1  which building answered you
$ curl -s https://www.cloudflare.com/cdn-cgi/trace | grep -E 'colo|tls|kex'
# 2  anycast: one IP, which city?
$ dig +short CHAOS TXT id.server @1.1.1.1       → "BOM"
# 3  the headers a CDN adds
$ curl -sI https://www.cloudflare.com/ | grep -iE 'cf-ray|cf-cache|age|alt-svc'
# 4  physics: same handshake, Mumbai vs Virginia (AWS regional endpoints)
$ curl -w 'tls %{time_appconnect}s\n' -so /dev/null \
      https://dynamodb.ap-south-1.amazonaws.com/ping   # vs us-east-1
# 5  put YOUR laptop behind the edge (install cloudflared first)
$ python3 tools/echo_server.py 8000  # terminal 1
$ cloudflared tunnel --url http://localhost:8000  → https://<rand>.trycloudflare.com
```

The quick-tunnel demo is the one to do together

That random URL puts your laptop behind Cloudflare's edge in seconds — no account, no DNS. Share it; the class opens it. tools/echo_server.py prints each request's CF-Connecting-IP, the CF-Ray with the visitor's colo, CDN-Loop — and how few TCP connections cloudflared uses for the whole room (that's the pool). Limits: 200 in-flight, no Server-Sent Events, testing only. Also open radar.cloudflare.com and speed.cloudflare.com in a browser.

DEMO · account

▶ The account demos — free plan, one zone. (Instructor drives; do a dry run.)

do this	in the dashboard	the point	free-plan limit
orange vs grey cloud	DNS → toggle proxy	dig shows origin IP vanish behind anycast	—
cache an HTML page	Cache → Cache Rules	MISS→HIT→purge→MISS; Age climbs	10 rules
block naive SQLi	Security → WAF custom rule	same bypasses as demos/waf.sh	5 rules, no regex
rate-limit /api/*	Security → Rate limiting	~20 pass then 429; per-colo counts	1 rule, 10 s only
strip X-Powered-By	Rules → Transform Rules	header hygiene, no code	unlimited
deploy a Worker	wrangler deploy	your code at the edge (next slide)	100k req/day

Free plan is enough to learn all of it — and the limits are themselves the lesson

1 WAF rule with no regex is why demos/waf.sh's bypasses matter; a 10-second-only rate window is exactly the fixed-window weakness from slide 22; HTML not cached until you add a Cache Rule is slide 11. The constraints teach the concepts. Full step-by-step, with the exact expressions to paste, is in cloudflare/INSTRUCTOR.md.

◂ S3: isn't everything a webserver?

▶ The edge runs your code — and it's Session 3's question, answered again

```
// worker-gateway: the API gateway demo, as a real Worker
export default {
  async fetch(request, env) {
    const { success } = await env.LIMITER.limit(
                              { key: ip });   // rate limit
    const claims = await verifyJwt(token,
                          env.JWT_SECRET);    // WebCrypto
    headers.set('X-User', claims.sub);
    return fetch(origin, { headers });       // primary→backup
  }
}   // verified in workerd: node test_workers.mjs
```

Session 3 asked: “isn't everything a web server?”

CGI forked a process per request; FastCGI kept them warm; nginx ran an event loop. A Worker is the next answer: a V8 isolate, not a process — starts in ~5 ms, ~3 MB, not the 35 MB + cold-start of a container.

amortise the setup, one more time

No fork+exec per request (Session 3's CGI cost); no VM to boot. The runtime is already resident in every PoP; your code just runs in it. It is prefork / FastCGI / servlets — the same closing idea — taken to the edge.

What actually ran here

Both Workers in cloudflare/ were executed locally in workerd (Cloudflare's real runtime, via Miniflare) against the lab origin. The gateway Worker reproduced every line of demos/gateway.sh — 401 on no token, alg:none refused, failover to backup. The ESI Worker's assembled page was checked byte-for-byte against the origin's own — and there's a framing bug in it (why not HTMLRewriter) that is Session 2's “where does this element end?” all over again. Story's in the README.

▶ What you hand over when you put a stranger in the middle

Your TLS private keys

The edge terminates HTTPS, so it holds the keys (or its own cert for your domain). It can read every byte. That is the deal.

A single point of failure

Fastly, 8 Jun 2021: one customer's config change → 85% of the network to errors → Reddit, Spotify, gov.uk down. 49 min to recover.

The power to see and shape traffic

Domain fronting used CDNs to hide censored traffic behind innocent domains — until AWS, Google (2018), Fastly (2024) all banned it. Whoever runs the edge decides what flows.

Lock-in by architecture

Once WAF, rate limits, auth and Workers live at the edge, it is not a cache you swap — it is your security perimeter.

A CDN is enormous leverage rented cheaply — and the rent is trust, concentration, and dependence. Use it. Know the bill. Next week is what to do when the thing in the middle fails.

Fastly post-incident report, Jun 2021 · domain-fronting bans: AWS/Google 2018, Azure 2022, Fastly Feb 2024

The four ideas

1 Move the work, or move the data

You cannot beat the speed of light, so you move content closer (the cache) or push personalisation outward (SSI→ESI→Ajax) until only the shared part travels far. Every CDN trick is one of these two moves.

2 The edge is wherever all the traffic converges

Caching routed everything through the edge; so the firewall, the rate limiter, the DDoS scrubber, the auth and now your code all followed the traffic there. Speed made you adopt it; control is why it's now your perimeter.

3 It's all Sessions 2–5, rented by the city

Reverse proxy (S4), connection pool (S3), ETag revalidation (S5), BBR vs the sawtooth (S2), multiplexing (S5), consistent hashing, framing (S2). A CDN invents almost nothing — it distributes what you already know.

4 Concentration cuts both ways

One network can absorb a 31 Tbps flood no one could survive alone — and one config bug in it takes 85% of the web down at once. The same pooling that is your strongest defence is your single point of failure.

cdn-lab — every number today came out of this, with no network

```
$ ./setup.sh      # a private CA + 3 certs. openssl only.
$ ./run_all.sh    # 7 benches + 6 demos, ~80 s, no network

origin/origin.py   the naive app server (trusts X-User, f-string SQL, leaks headers)
edge/edge.py       a CDN PoP in one file: cache · pool · WAF · rate limit · JWT · ESI
tools/laggy.py     a TCP proxy that adds distance (the physics floor)
cloudflare/        no-account.sh · INSTRUCTOR.md · two Workers (run in workerd)
```

Honest about what it does NOT model

laggy.py delays, it never drops — no loss, no bandwidth (BBR vs CUBIC needs tc netem, homework 2). Every hop is HTTP/1.1. Anycast and BGP can't live on one machine — use cloudflare/no-account.sh for those. The worker-pool bench assigns each visitor a random simulated worker rather than a real kernel accept queue. Where a number is a model and not a measurement, the slide and the README say so.

Read edge/edge.py end to end — it is ~450 lines and it is the whole lecture as code. Then break it: set --pool 0 and watch reuse collapse, --waf off and watch the injection land.

07 · ASSIGNMENT

▶ Put something real behind the edge, and prove what it did

Build a small origin, front it with Cloudflare, and measure the edge

Any language. Serve one cacheable static asset and one personalised HTML page. Put it behind Cloudflare (a free zone, or a Quick Tunnel — no account, no domain needed). Then make the edge prove itself with numbers, not screenshots.

Cache

Show cf-cache-status going MISS→HIT for the asset, and DYNAMIC for the page. Then make the page cacheable and show the difference in origin request count (log it at the origin).

The key

Demonstrate one cache-key hazard on your own site — a header your origin reflects but the key ignores — then close it. Write up why it worked.

Measure

Report TTFB from far away (a cloud box in another region, or a friend) for cold origin vs warm edge vs cache HIT. One table, like bench/02.

One edge feature

Add exactly one: a WAF rule, a rate limit, or a Worker that verifies a token. Show a request it blocks and one it allows.

Deliverable: the code, plus a one-page write-up with the numbers and the cache-key story. Due before Session 7.

▸ Homework

1. **Run the whole lab and read edge/edge.py.** ./setup.sh && ./run_all.sh, then read the edge end to end. Change one flag (--pool 0, --waf off, --strip-unkeyed) and explain the number that moves.

2. **Redo BBR vs CUBIC for real.** laggy.py only delays. On a real Linux box, use tc netem to add 1% loss at 150 ms RTT, download a large file under cubic and under bbr (sysctl), and report the gap. This is the number bench/05 refused to fake.

3. **Poison your own cache, then fix it three ways.** Reproduce demos/poison.sh against a site you control. Close it three different ways — drop the header, add it to the key, Vary on it — and argue which a real CDN should default to, and why.

4. **Find a real cf-cache-status in the wild.** curl -I ten sites you use. For each, explain its cache status, its Cache-Control, and what its cache key must be. Find one HIT you think is a mistake.

5. **The one to do if you do only one: measure the speed of light to your users.** Pick your origin's real region. For 5 cities your users are in, compute the RTT floor (bench/01) and measure the real ping. Where is the gap biggest — and would a CDN close it?

Number 5 is the one to do if you only do one — it makes the physics personal.

NEXT · WEEK 7

Building for failures

You just put a stranger in the middle of every request, and pooled your fate with everyone else on it. Now: what happens when it breaks?

The threads today leaves hanging

- stale-if-error — today it was a footnote; next week the cache is the last thing standing when the origin dies
- the Fastly 49 minutes, and the retry/timeout/circuit-breaker logic that decides who notices an outage and who doesn't
- failover (demos/failover.sh: primary→backup, we watched it) · feature flags · graceful degradation · the thundering herd on recovery

The question to sit with: today's edge survives your origin dying — a stale HIT. What does your origin do when the edge, or a whole region, is the thing that dies?